Demo: From Observed Treasury Yields to Hull–White Future Curves

A hands-on companion to Lecture 7

Table of contents

1 Goal

This demo follows one complete fixed-income workflow:

1. start with a **dated U.S. Treasury par-yield snapshot**;
2. bootstrap today's discount factors and zero-coupon yield curve;
3. calibrate Hull–White so that it exactly respects that curve;
4. fit the model's mean reversion and volatility to a supplied synthetic option quote snapshot that demonstrates the calibration mechanics;
5. generate yield curves at a future date in different interest-rate states; and
6. use those future curves to price remaining bond cash flows.

The initial curve uses the U.S. Treasury's published **January 2, 2025** par-yield snapshot. To keep every calculation auditable, the market nodes are embedded below rather than downloaded during rendering. Treasury par yields are bond-equivalent quoted rates; bootstrapped zero rates are continuously compounded. The option premiums used later are explicitly labeled synthetic because a stable, public option-volatility surface is not available with the course files.

Source: U.S. Treasury, Daily Treasury Par Yield Curve Rates.

> **The central idea**
>
> Today's curve determines today's prices. Hull–White adds a disciplined way to describe *future* curves. Calibration makes those two statements consistent.

2 Setup

We only need NumPy, pandas, and Matplotlib. The functions below use explicit inputs and return ordinary arrays or data frames so that each step is easy to inspect.

```
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import display

np.random.seed(411)

# A restrained, consistent plotting style.
BLUE = "#2C6EAA"
```

```
GOLD = "#D4A72C"
INK = "#263238"
LIGHT_BLUE = "#B9D6EC"
GRID = "#D9E1E8"

plt.rcParams.update({
    "figure.figsize": (8, 4.6),
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.alpha": 0.65,
    "grid.color": GRID,
    "axes.titleweight": "bold",
})
```

3 Step 1: Record the observed Treasury curve

The published Treasury series contains par yields at standard maturity nodes. We retain the 1-, 2-, 3-, 5-, 7-, and 10-year observations and linearly interpolate missing integer maturities. The interpolation is an **estimated curve-construction choice**, not an additional market quote. For transparent annual bootstrapping we treat each interpolated par yield as the coupon on an annual-pay par bond. A production Treasury curve would instead use security-level prices, semiannual coupons, actual day counts, accrued interest, and bills at the short end.

```
def price_annual_coupon_bond(coupon_rate, maturity, discount_factors, face=100.0):
    """Price an annual-pay bond from discount factors at integer years.

    discount_factors[k - 1] must equal P(0, k).
    """
    coupon = face * coupon_rate
    cash_flows = np.full(maturity, coupon)
    cash_flows[-1] += face
    return float(np.sum(cash_flows * discount_factors[:maturity]))


observed_date = "2025-01-02"
observed_maturities = np.array([1, 2, 3, 5, 7, 10], dtype=float)
observed_par_yields = np.array([4.17, 4.25, 4.29, 4.38, 4.47, 4.57]) / 100

market_nodes = pd.DataFrame({
    "observation_date": observed_date,
```

```
    "maturity_years": observed_maturities,
    "observed_par_yield_pct": 100 * observed_par_yields,
})
display(market_nodes)

maturities = np.arange(1, 11)
coupon_rates = np.interp(maturities, observed_maturities, observed_par_yields)

# Under the annual-pay teaching convention, a par-yield quote implies price 100.
clean_prices = np.full(len(maturities), 100.0)

bond_market = pd.DataFrame({
    "maturity_years": maturities,
    "par_yield_pct": 100 * coupon_rates,
    "implied_par_price": clean_prices,
    "input_status": np.where(np.isin(maturities, observed_maturities),
                             "Observed Treasury node", "Interpolated node"),
})

display(bond_market)
```

	observation_date	maturity_years	observed_par_yield_pct
0	2025-01-02	1.0	4.17
1	2025-01-02	2.0	4.25
2	2025-01-02	3.0	4.29
3	2025-01-02	5.0	4.38
4	2025-01-02	7.0	4.47
5	2025-01-02	10.0	4.57

	maturity_years	par_yield_pct	implied_par_price	input_status
0	1	4.1700	100.0	Observed Treasury node
1	2	4.2500	100.0	Observed Treasury node
2	3	4.2900	100.0	Observed Treasury node
3	4	4.3350	100.0	Interpolated node
4	5	4.3800	100.0	Observed Treasury node
5	6	4.4250	100.0	Interpolated node
6	7	4.4700	100.0	Observed Treasury node
7	8	4.5033	100.0	Interpolated node
8	9	4.5367	100.0	Interpolated node

	maturity_years	par_yield_pct	implied_par_price	input_status
9	10	4.5700	100.0	Observed Treasury node

Observed nodes are facts from the dated snapshot. Interpolated nodes and the annual-pay par-bond representation are assumptions. That separation matters: a curve can fit its inputs perfectly while still depending on construction choices.

4 Step 2: Bootstrap today's discount curve

For the one-year bond, the only cash flow is coupon plus principal, so its price immediately gives $P(0,1)$. For each later bond, discount factors for earlier coupons are already known. The final discount factor is the only unknown:

$$P(0,n) = \frac{100 - \sum_{i=1}^{n-1} C_n P(0,i)}{100 + C_n}, \qquad C_n = 100 c_n.$$

```
def bootstrap_discount_factors(maturities, coupon_rates, bond_prices, face=100.0):
    """Bootstrap annual discount factors from annual-pay coupon bonds.

    Inputs
    ------
    maturities : array-like of integers
        Consecutive maturities 1, 2, ..., N.
    coupon_rates : array-like
        Annual coupon rates in decimal form.
    bond_prices : array-like
        Observed clean prices per 100 of face value.

    Output
    ------
    NumPy array
        Discount factors P(0, 1), ..., P(0, N).
    """
    maturities = np.asarray(maturities, dtype=int)
    coupon_rates = np.asarray(coupon_rates, dtype=float)
    bond_prices = np.asarray(bond_prices, dtype=float)

    expected = np.arange(1, len(maturities) + 1)
    if not np.array_equal(maturities, expected):
```

```
        raise ValueError("Maturities must be consecutive integers starting at 1.")

    discount_factors = np.empty(len(maturities))

    for row, maturity in enumerate(maturities):
        coupon = face * coupon_rates[row]

        # Present value of coupons before the maturity date.
        earlier_coupon_pv = coupon * discount_factors[:row].sum()

        # Solve the bond-pricing equation for the one remaining unknown.
        discount_factors[row] = (
            bond_prices[row] - earlier_coupon_pv
        ) / (face + coupon)

    return discount_factors


bootstrapped_discounts = bootstrap_discount_factors(
    maturities,
    coupon_rates,
    clean_prices,
)
bootstrapped_zero_rates = -np.log(bootstrapped_discounts) / maturities

curve_table = pd.DataFrame({
    "maturity_years": maturities,
    "discount_factor": bootstrapped_discounts,
    "zero_rate_pct": 100 * bootstrapped_zero_rates,
})
display(curve_table.round(6))
```

	maturity_years	discount_factor	zero_rate_pct
0	1	0.9600	4.0854
1	2	0.9201	4.1638
2	3	0.8815	4.2033
3	4	0.8437	4.2487
4	5	0.8068	4.2948
5	6	0.7707	4.3417
6	7	0.7355	4.3895
7	8	0.7019	4.4250

	maturity_years	discount_factor	zero_rate_pct
8	9	0.6693	4.4613
9	10	0.6377	4.4984

```
fig, axes = plt.subplots(1, 2, figsize=(10.5, 4.1))

axes[0].plot(maturities, bootstrapped_discounts, "o-", color=BLUE)
axes[0].set(title="Bootstrapped discount factors",
            xlabel="Maturity (years)", ylabel="$P(0,T)$")

axes[1].plot(maturities, 100 * bootstrapped_zero_rates, "o-", color=BLUE)
axes[1].set(title="Bootstrapped zero-coupon curve",
            xlabel="Maturity (years)", ylabel="Continuous zero rate (%)")

plt.tight_layout()
plt.show()
```

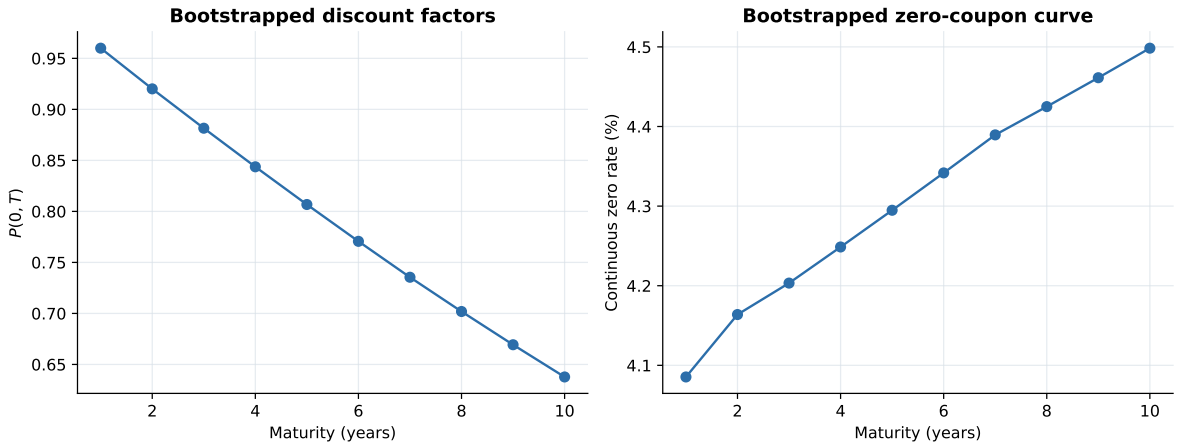


Figure 1: Today's curve recovered from observable coupon-bond prices

4.1 Check the bootstrap by repricing every input bond

A useful calibration habit is to return to the original observable quantities.

```
repriced_bonds = np.array([
    price_annual_coupon_bond(coupon, int(maturity), bootstrapped_discounts)
    for coupon, maturity in zip(coupon_rates, maturities)
```

```
])

bootstrap_check = bond_market.copy()
bootstrap_check["repriced_from_curve"] = repriced_bonds
bootstrap_check["pricing_error"] = repriced_bonds - clean_prices
display(bootstrap_check.round(8))

print(f"Largest absolute repricing error: "
      f"{np.max(np.abs(repriced_bonds - clean_prices)):.10f} per 100 face")
```

	maturity_years	par_yield_pct	implied_par_price	input_status	repriced_from_curve
0	1	4.1700	100.0	Observed Treasury node	100.0
1	2	4.2500	100.0	Observed Treasury node	100.0
2	3	4.2900	100.0	Observed Treasury node	100.0
3	4	4.3350	100.0	Interpolated node	100.0
4	5	4.3800	100.0	Observed Treasury node	100.0
5	6	4.4250	100.0	Interpolated node	100.0
6	7	4.4700	100.0	Observed Treasury node	100.0
7	8	4.5033	100.0	Interpolated node	100.0
8	9	4.5367	100.0	Interpolated node	100.0
9	10	4.5700	100.0	Observed Treasury node	100.0

```
Largest absolute repricing error: 0.0000000000 per 100 face
```

5 Step 3: Make the initial curve usable at any maturity

Hull–White formulas require $P(0,T)$ and the instantaneous forward rate $f(0,t)$. We use linear interpolation of the continuously compounded zero rate. This is intentionally simple; production systems use richer curve construction.

```
def zero_rate_0(maturity):
    """Interpolate today's bootstrapped continuous zero rate."""
    maturity = np.asarray(maturity, dtype=float)
    return np.interp(
        maturity,
        np.r_[0.0, maturities],
        np.r_[bootstrapped_zero_rates[0], bootstrapped_zero_rates],
    )
```

```python
def discount_0(maturity):
    """Return today's discount factor P(0,T)."""
    maturity = np.asarray(maturity, dtype=float)
    return np.exp(-zero_rate_0(maturity) * maturity)


def forward_rate_0(time, step=1e-4):
    """Approximate f(0,t) = -d log(P(0,t)) / dt numerically."""
    time = np.asarray(time, dtype=float)
    left = np.maximum(time - step, 0.0)
    right = time + step
    return -(np.log(discount_0(right)) - np.log(discount_0(left))) / (right - left)
```

6 Step 4: Fit the Hull–White dynamics to option prices

The lecture's calibration logic separates two jobs:

- the function $\theta(t)$ makes the model fit today's *entire curve*; and
- a and σ control future mean reversion and volatility.

In practice, a and σ are fitted to liquid caps, floors, or swaptions. To keep the code transparent, our artificial option market consists of European calls on zero-coupon bonds. Such option premiums are observable, and Hull–White has a short closed-form pricing formula for them.

6.1 Hull–White building blocks

For the one-factor model

$$dr_t = [\theta(t) - ar_t]dt + \sigma dW_t,$$

define

$$B(t,T) = \frac{1 - e^{-a(T-t)}}{a}.$$

The volatility of a zero-coupon bond option expiring at S on a bond maturing at T is

$$\sigma_P = \sigma B(S,T)\sqrt{\frac{1 - e^{-2aS}}{2a}}.$$

```
def normal_cdf(x):
    """Standard normal cumulative probability; works for scalars."""
    return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))


def hw_B(start, end, mean_reversion):
    """Return the Hull-White B(start, end) loading."""
    return (1.0 - np.exp(-mean_reversion * (end - start))) / mean_reversion


def zero_bond_call_price(expiry, bond_maturity, strike,
                         mean_reversion, volatility, face=100.0):
    """Price a European call on a zero-coupon bond under Hull-White.

    The option expires at `expiry`. Its underlying pays `face` at
    `bond_maturity`. The strike is paid at expiry per `face` of principal.
    """
    p_0_s = float(discount_0(expiry))
    p_0_t = float(discount_0(bond_maturity))

    bond_vol = (
        volatility
        * hw_B(expiry, bond_maturity, mean_reversion)
        * math.sqrt((1.0 - math.exp(-2.0 * mean_reversion * expiry))
                    / (2.0 * mean_reversion))
    )

    # Work with values per $1 face, then multiply by face at the end.
    strike_per_dollar = strike / face
    h = (
        math.log(p_0_t / (strike_per_dollar * p_0_s)) / bond_vol
        + 0.5 * bond_vol
    )
    call_per_dollar = (
        p_0_t * normal_cdf(h)
        - strike_per_dollar * p_0_s * normal_cdf(h - bond_vol)
    )
    return face * call_per_dollar
```

6.2 Create a synthetic option quote snapshot

We create at-the-money-forward calls with expiries from six months to five years. The hidden values $a = 0.18$ and $\sigma = 1.20\%$ are used only to generate the market premiums. A small alternating bid/ask-like perturbation prevents a mechanically perfect parameter recovery.

```
option_expiries = np.array([0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0])
underlying_maturities = option_expiries + 5.0

# An ATM-forward strike makes the option comparison especially clear.
option_strikes = np.array([
    100.0 * discount_0(maturity) / discount_0(expiry)
    for expiry, maturity in zip(option_expiries, underlying_maturities)
])

hidden_a = 0.18
hidden_sigma = 0.012
quote_perturbation = np.array([1.003, 0.998, 1.002, 0.997, 1.001, 0.999, 1.002])

observed_option_prices = np.array([
    zero_bond_call_price(expiry, maturity, strike, hidden_a, hidden_sigma)
    for expiry, maturity, strike in zip(
        option_expiries, underlying_maturities, option_strikes
    )
]) * quote_perturbation

option_market = pd.DataFrame({
    "option_expiry": option_expiries,
    "bond_maturity": underlying_maturities,
    "strike_per_100": option_strikes,
    "observed_option_price": observed_option_prices,
})
display(option_market.round(5))
```

	option_expiry	bond_maturity	strike_per_100	observed_option_price
0	0.5	5.5	80.4869	0.8444
1	1.0	6.0	80.2801	1.1124
2	1.5	6.5	80.1000	1.2818
3	2.0	7.0	79.9326	1.3817
4	3.0	8.0	79.6205	1.5017
5	4.0	9.0	79.3289	1.5362

	option_expiry	bond_maturity	strike_per_100	observed_option_price
6	5.0	10.0	79.0493	1.5355

6.3 Calibrate a and σ with a simple grid search

The objective is the root-mean-square pricing error across the option quotes. A grid search is slower than a numerical optimizer, but students can see exactly what is tried and it needs no extra package.

```
def option_rmse(mean_reversion, volatility):
    """Return RMSE between Hull-White and observed option prices."""
    model_prices = np.array([
        zero_bond_call_price(expiry, maturity, strike,
                             mean_reversion, volatility)
        for expiry, maturity, strike in zip(
            option_expiries, underlying_maturities, option_strikes
        )
    ])
    return float(np.sqrt(np.mean((model_prices - observed_option_prices) ** 2)))


a_grid = np.linspace(0.03, 0.40, 150)
sigma_grid = np.linspace(0.006, 0.020, 150)

# Each matrix cell is the pricing error for one candidate (a, sigma) pair.
error_surface = np.array([
    [option_rmse(a, sigma) for sigma in sigma_grid]
    for a in a_grid
])

best_row, best_col = np.unravel_index(np.argmin(error_surface), error_surface.shape)
calibrated_a = a_grid[best_row]
calibrated_sigma = sigma_grid[best_col]

print(f"Calibrated mean reversion a: {calibrated_a:.4f} per year")
print(f"Calibrated volatility sigma: {100 * calibrated_sigma:.4f}% per sqrt(year)")
print(f"Option-price RMSE: {error_surface[best_row, best_col]:.6f} per 100 face")
```

```
Calibrated mean reversion a: 0.1790 per year
Calibrated volatility sigma: 1.1919% per sqrt(year)
Option-price RMSE: 0.005442 per 100 face
```

```
fig, ax = plt.subplots(figsize=(8, 4.8))
contour = ax.contourf(
    100 * sigma_grid,
    a_grid,
    np.log10(error_surface + 1e-8),
    levels=25,
    cmap="Blues_r",
)
ax.scatter(100 * calibrated_sigma, calibrated_a, s=90, color=GOLD,
           edgecolor=INK, label="Best grid point", zorder=3)
ax.set(xlabel="$\sigma$ (% per $\sqrt{\mathrm{year}}$)",
       ylabel="$a$ (per year)",
       title="Hull-White calibration objective")
ax.legend()
fig.colorbar(contour, ax=ax, label="log10(option-price RMSE)")
plt.show()
```

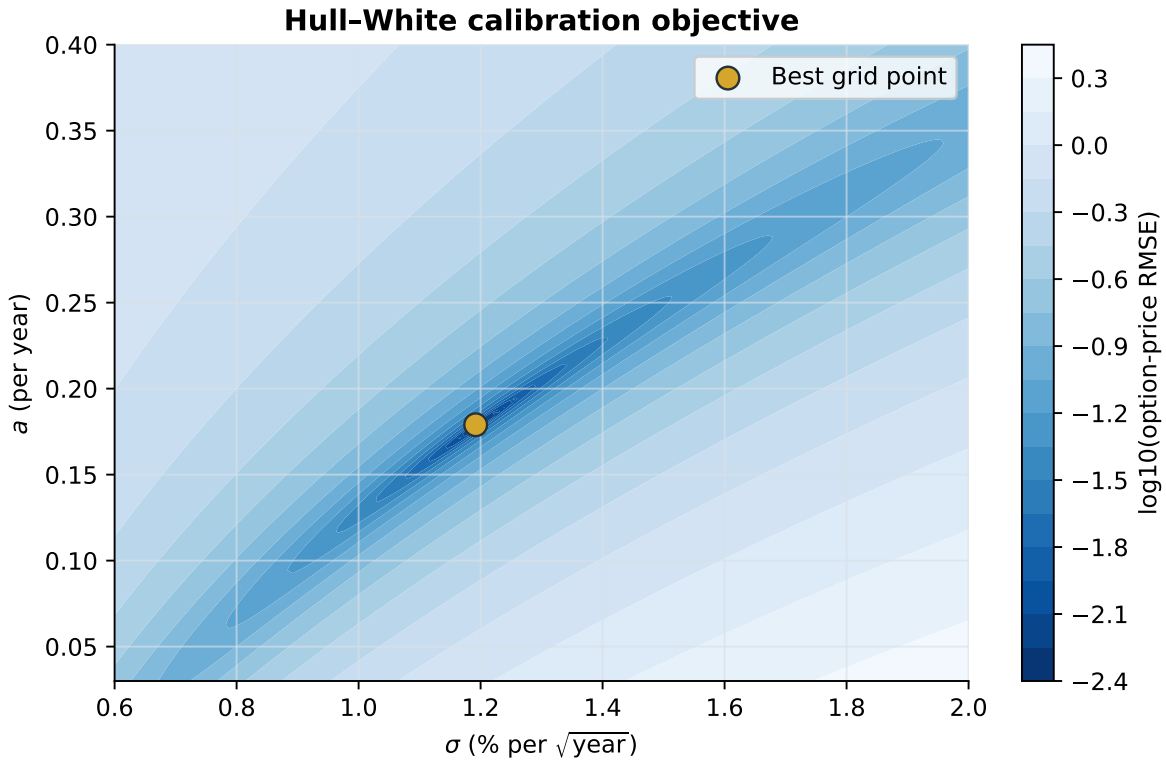


Figure 2: Option-pricing error across candidate Hull–White parameters

```
fitted_option_prices = np.array([
    zero_bond_call_price(expiry, maturity, strike,
                         calibrated_a, calibrated_sigma)
    for expiry, maturity, strike in zip(
        option_expiries, underlying_maturities, option_strikes
    )
])

option_fit = option_market.copy()
option_fit["model_price"] = fitted_option_prices
option_fit["pricing_error"] = fitted_option_prices - observed_option_prices
display(option_fit.round(6))
```

	option_expiry	bond_maturity	strike_per_100	observed_option_price	model_price	pricing_error
0	0.5	5.5	80.4869	0.8444	0.8383	-0.0062
1	1.0	6.0	80.2801	1.1124	1.1100	-0.0023
2	1.5	6.5	80.1000	1.2818	1.2743	-0.0075
3	2.0	7.0	79.9326	1.3817	1.3807	-0.0010
4	3.0	8.0	79.6205	1.5017	1.4952	-0.0065
5	4.0	9.0	79.3289	1.5362	1.5331	-0.0031
6	5.0	10.0	79.0493	1.5355	1.5281	-0.0073

```
plt.plot(option_expiries, observed_option_prices, "o-", color=BLUE,
         label="Observed option premiums")
plt.plot(option_expiries, fitted_option_prices, "s--", color=GOLD,
         label="Hull-White fitted premiums")
plt.xlabel("Option expiry (years)")
plt.ylabel("Option price per 100 face")
plt.title("Fit to the option market")
plt.legend()
plt.show()
```

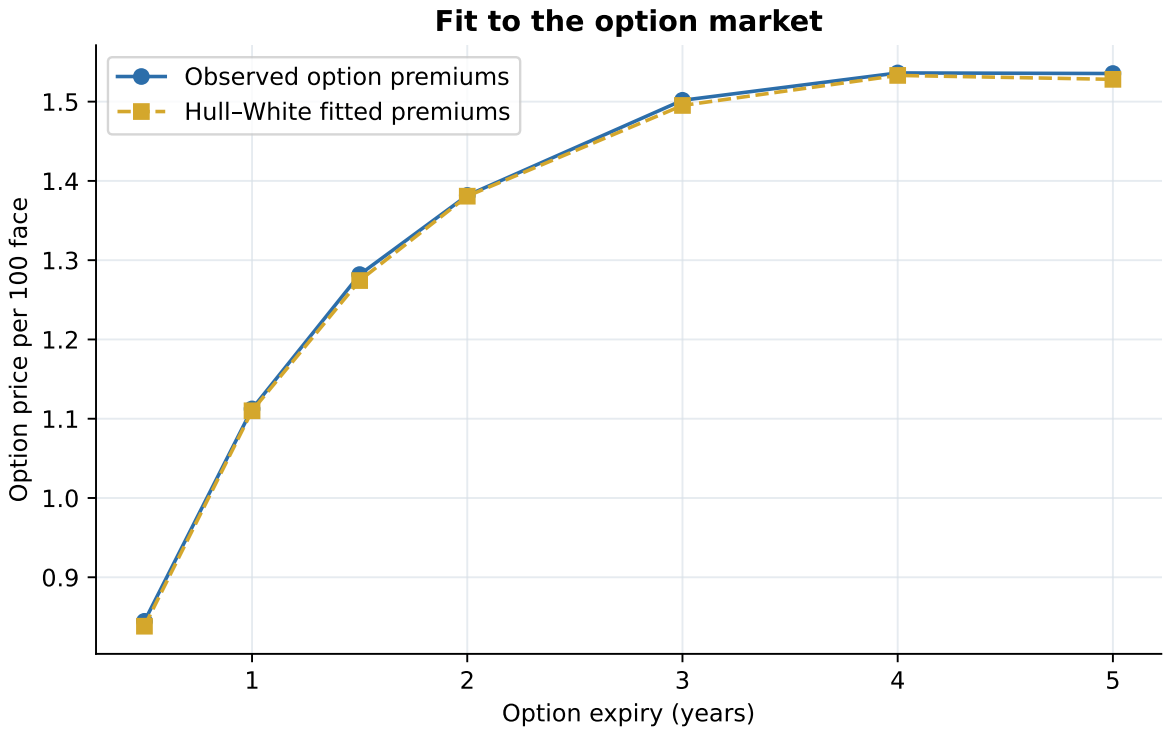


Figure 3: Observed artificial option premiums and calibrated Hull–White prices

7 Step 5: Fit the time-dependent drift to today's curve

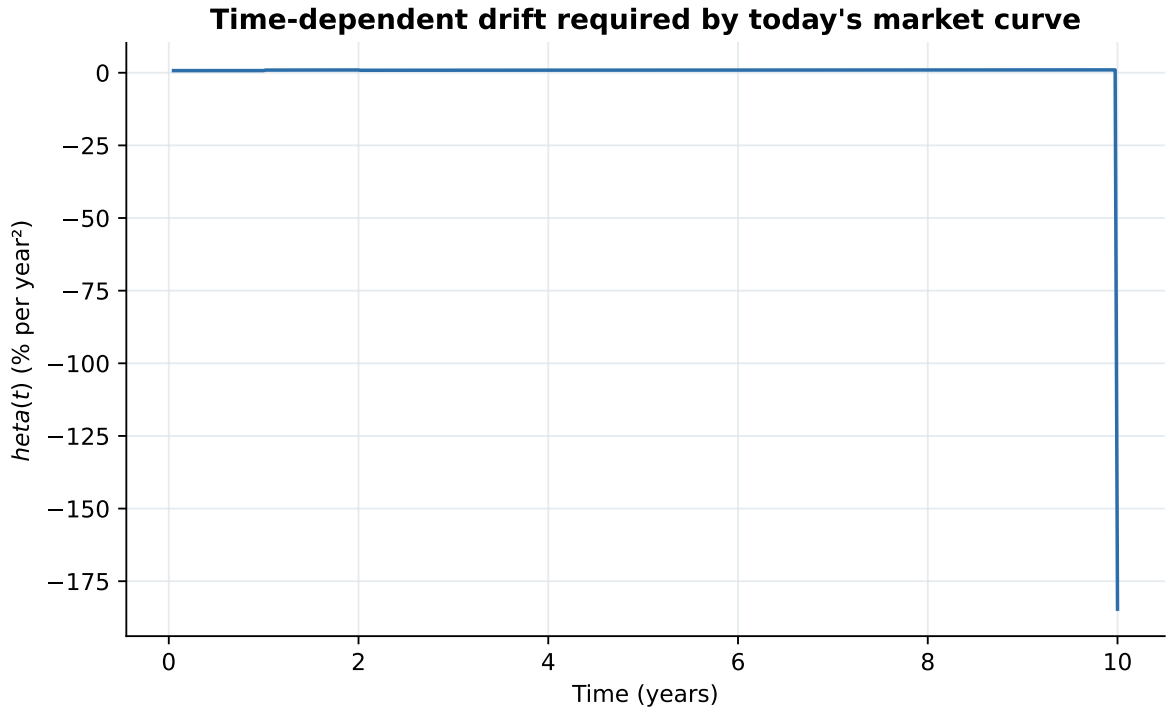
Once a and σ are known, the drift function that fits the observed initial curve is

$$\theta(t) = \frac{\partial f(0,t)}{\partial t} + a f(0,t) + \frac{\sigma^2}{2a}\left(1 - e^{-2at}\right).$$

This is the lecture's key calibration point: $\theta(t)$ is a *function*, not one number. It supplies enough flexibility to match all maturities of today's curve.

```
def theta(time, mean_reversion, volatility, step=1e-3):
    """Return the Hull-White drift theta(t) fitted to today's curve."""
    time = np.asarray(time, dtype=float)
    left = np.maximum(time - step, 0.0)
    right = time + step

    # Numerical derivative of today's instantaneous forward curve.
```

```
    forward_slope = (
        forward_rate_0(right) - forward_rate_0(left)
    ) / (right - left)

    return (
        forward_slope
        + mean_reversion * forward_rate_0(time)
        + volatility**2 / (2.0 * mean_reversion)
          * (1.0 - np.exp(-2.0 * mean_reversion * time))
    )


theta_times = np.linspace(0.05, 10.0, 400)
theta_values = theta(theta_times, calibrated_a, calibrated_sigma)

plt.plot(theta_times, 100 * theta_values, color=BLUE)
plt.xlabel("Time (years)")
plt.ylabel("$\theta(t)$ (% per year²)")
plt.title("Time-dependent drift required by today's market curve")
plt.show()
```



The sharp changes are a consequence of using simple piecewise-linear interpolation. That is instructive: curve construction and smoothing matter in real calibration.

7.1 Verify that Hull–White reproduces today's discount curve

For future-date pricing, Hull–White gives the zero-coupon bond price

$$P(t,T) = A(t,T)e^{-B(t,T)r_t},$$

where

$$A(t,T) = \frac{P(0,T)}{P(0,t)} \exp\left[B(t,T)f(0,t) - \frac{\sigma^2}{4a}(1-e^{-2at})B(t,T)^2\right].$$

At $t = 0$, using $r_0 = f(0,0)$ makes the exponential terms cancel, leaving exactly $P(0,T)$. This is the no-arbitrage fit.

```
def hw_discount_factor(valuation_time, maturity, short_rate,
                       mean_reversion, volatility):
    """Return the Hull-White state-dependent discount factor P(t,T).

    Inputs
    ------
    valuation_time : float
        Future valuation date t, in years from today.
    maturity : number or array
        Payment date(s) T, with T > t.
    short_rate : float
        Short rate r_t observed or simulated at the valuation date.
    mean_reversion, volatility : float
        Calibrated Hull-White parameters a and sigma.
    """
    maturity = np.asarray(maturity, dtype=float)
    b_value = hw_B(valuation_time, maturity, mean_reversion)

    variance_adjustment = (
        volatility**2
        / (4.0 * mean_reversion)
        * (1.0 - np.exp(-2.0 * mean_reversion * valuation_time))
        * b_value**2
    )
```

```
    a_value = (
        discount_0(maturity) / discount_0(valuation_time)
        * np.exp(
            b_value * forward_rate_0(valuation_time)
            - variance_adjustment
        )
    )
    return a_value * np.exp(-b_value * short_rate)


r_0 = float(forward_rate_0(0.0))
hw_discounts_today = hw_discount_factor(
    0.0, maturities, r_0, calibrated_a, calibrated_sigma
)

initial_fit = pd.DataFrame({
    "maturity": maturities,
    "market_discount": bootstrapped_discounts,
    "hw_discount": hw_discounts_today,
    "difference": hw_discounts_today - bootstrapped_discounts,
})
display(initial_fit.round(10))
```

	maturity	market_discount	hw_discount	difference
0	1	0.9600	0.9600	0.0
1	2	0.9201	0.9201	0.0
2	3	0.8815	0.8815	-0.0
3	4	0.8437	0.8437	0.0
4	5	0.8068	0.8068	0.0
5	6	0.7707	0.7707	0.0
6	7	0.7355	0.7355	0.0
7	8	0.7019	0.7019	0.0
8	9	0.6693	0.6693	0.0
9	10	0.6377	0.6377	0.0

8 Step 6: Generate future yield curves

Move to year 2. In a one-factor Hull–White model, the short rate r_2 is the single state variable. Once a state is specified, `hw_discount_factor()` produces $P(2, T)$ for every $T > 2$,

and therefore the entire future zero curve:

$$y(2,T) = -\frac{\log P(2,T)}{T-2}.$$

We show three possible states around the model's central short-rate level. They are scenarios, not point forecasts.

```
future_time = 2.0
future_maturities = np.linspace(2.25, 10.0, 80)
central_short_rate = float(forward_rate_0(future_time))

short_rate_states = {
    "Low-rate state": central_short_rate - 0.015,
    "Central state": central_short_rate,
    "High-rate state": central_short_rate + 0.015,
}

future_curve_rows = []
for state_name, short_rate in short_rate_states.items():
    state_discounts = hw_discount_factor(
        future_time,
        future_maturities,
        short_rate,
        calibrated_a,
        calibrated_sigma,
    )
    state_yields = -np.log(state_discounts) / (future_maturities - future_time)

    future_curve_rows.append(pd.DataFrame({
        "state": state_name,
        "short_rate": short_rate,
        "maturity": future_maturities,
        "zero_rate": state_yields,
    }))

future_curves = pd.concat(future_curve_rows, ignore_index=True)
display(future_curves.groupby("state").head(3).round(5))
```

	state	short_rate	maturity	zero_rate
0	Low-rate state	0.0278	2.2500	0.0279

	state	short_rate	maturity	zero_rate
1	Low-rate state	0.0278	2.3481	0.0281
2	Low-rate state	0.0278	2.4462	0.0282
80	Central state	0.0428	2.2500	0.0425
81	Central state	0.0428	2.3481	0.0426
82	Central state	0.0428	2.4462	0.0427
160	High-rate state	0.0578	2.2500	0.0572
161	High-rate state	0.0578	2.3481	0.0571
162	High-rate state	0.0578	2.4462	0.0571

```
styles = {
    "Low-rate state": (BLUE, "-"),
    "Central state": (INK, "--"),
    "High-rate state": (GOLD, "-."),
}

for state_name, group in future_curves.groupby("state", sort=False):
    color, line_style = styles[state_name]
    plt.plot(group["maturity"] - future_time, 100 * group["zero_rate"],
             color=color, linestyle=line_style, linewidth=2,
             label=f"{state_name} (r  = {100 * group['short_rate'].iloc[0]:.2f}%)")

plt.xlabel("Remaining maturity, $T-2$ (years)")
plt.ylabel("Year-2 zero rate (%)")
plt.title("Hull-White yield curves at a future date")
plt.legend()
plt.show()
```

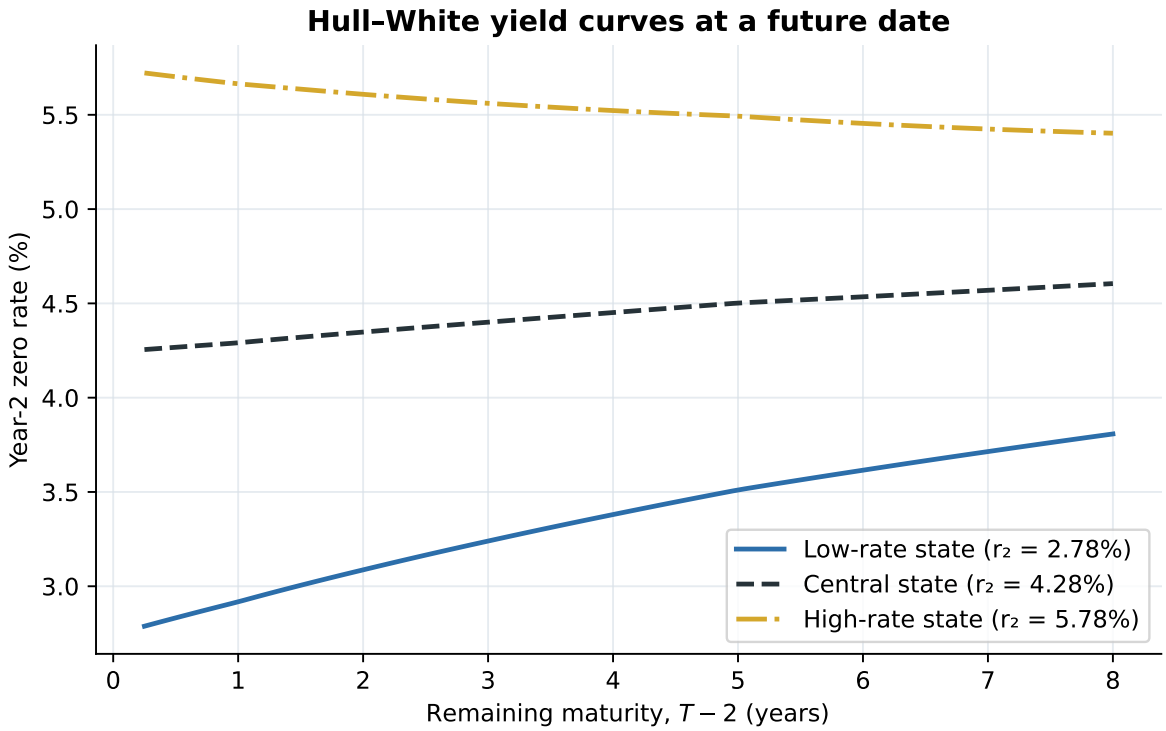


Figure 4: At year 2, one short-rate state determines a complete remaining yield curve

Notice that one factor moves the whole curve in a structured way. Mean reversion causes the effect of today's short-rate state to weaken at longer maturities because $B(t, T)$ approaches $1/a$ rather than increasing without bound.

9 Step 7: Use a future curve to price a bond

Consider the original ten-year 4.10% coupon bond at year 2, immediately after its year-2 coupon has been paid. Eight coupons and the principal remain. Its value in each state is simply the discounted value of those remaining cash flows using $P(2, T)$.

```
def price_bond_at_time(valuation_time, payment_times, cash_flows,
                       short_rate, mean_reversion, volatility):
    """Price fixed cash flows at a future date using a Hull-White curve.

    Output is the value at `valuation_time`, not today's value.
    """
    payment_times = np.asarray(payment_times, dtype=float)
```

```
    cash_flows = np.asarray(cash_flows, dtype=float)

    if np.any(payment_times <= valuation_time):
        raise ValueError("Every payment must occur after the valuation time.")

    future_discounts = hw_discount_factor(
        valuation_time,
        payment_times,
        short_rate,
        mean_reversion,
        volatility,
    )
    return float(np.sum(cash_flows * future_discounts))


payment_times = np.arange(3.0, 11.0)
remaining_cash_flows = np.full(len(payment_times), 4.10)
remaining_cash_flows[-1] += 100.0

future_bond_values = []
for state_name, short_rate in short_rate_states.items():
    value = price_bond_at_time(
        future_time,
        payment_times,
        remaining_cash_flows,
        short_rate,
        calibrated_a,
        calibrated_sigma,
    )
    future_bond_values.append({
        "state": state_name,
        "short_rate_pct": 100 * short_rate,
        "bond_value_at_year_2": value,
    })

future_value_table = pd.DataFrame(future_bond_values)
display(future_value_table.round(4))
```

	state	short_rate_pct	bond_value_at_year_2
0	Low-rate state	2.7817	101.8046
1	Central state	4.2817	96.1060

	state	short_rate_pct	bond_value_at_year_2
2	High-rate state	5.7817	90.7425

```
plot_table = future_value_table.sort_values("short_rate_pct")
bars = plt.bar(plot_table["state"], plot_table["bond_value_at_year_2"],
               color=[BLUE, LIGHT_BLUE, GOLD], edgecolor=INK)
plt.bar_label(bars, fmt="%.2f", padding=3)
plt.ylabel("Bond value at year 2 per 100 face")
plt.title("Future bond value by Hull-White state")
plt.ylim(0, 1.14 * plot_table["bond_value_at_year_2"].max())
plt.show()
```

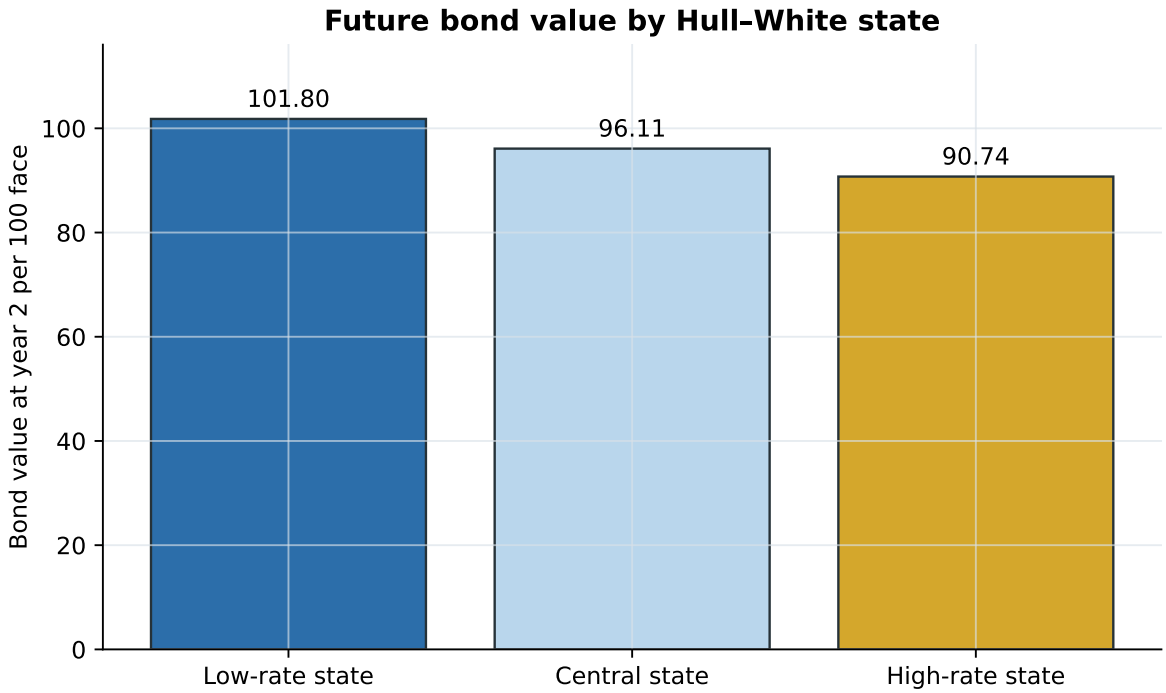


Figure 5: The same promised cash flows have different values in different year-2 rate states

This future valuation is where a short-rate model becomes useful. Today we do not observe $P(2, T)$; Hull–White generates it consistently for each possible r_2.

10 Step 8: Plain-bond consistency at time 0

Finally, verify the lecture's important claim: once Hull–White is calibrated to the observed curve, it does **not** create a different time-0 price for a vanilla bond. Both methods discount the same fixed cash flows with the same $P(0,T)$.

```
vanilla_payment_times = np.arange(1.0, 11.0)
vanilla_cash_flows = np.full(10, 4.10)
vanilla_cash_flows[-1] += 100.0

# Method 1: discount directly with today's observed/bootstrapped curve.
direct_curve_price = float(np.sum(
    vanilla_cash_flows * discount_0(vanilla_payment_times)
))

# Method 2: use the calibrated Hull-White zero-coupon formula at t = 0.
hw_curve_price = float(np.sum(
    vanilla_cash_flows
    * hw_discount_factor(
        0.0,
        vanilla_payment_times,
        r_0,
        calibrated_a,
        calibrated_sigma,
    )
))

print(f"Direct observed-curve price: {direct_curve_price:.10f}")
print(f"Calibrated Hull-White price:  {hw_curve_price:.10f}")
print(f"Difference:                    {hw_curve_price - direct_curve_price:.12f}")
```

```
Direct observed-curve price: 96.2742663687
Calibrated Hull-White price:  96.2742663687
Difference:                    0.000000000000
```

The equality is structural, not a coincidence. At time 0, the calibrated model uses the market discount factors by construction. For a vanilla bond, the model adds no new pricing information. Its value is the ability to generate future curves and to value cash flows that depend on those future states.

11 Checks and interpretation

The workflow has three separate fits, each with a different purpose:

Calibration object	Observable input	What is fitted	Why it matters
Today's discount curve	Coupon-bond prices	$P(0,T)$	Prices fixed cash flows today
Initial Hull–White term structure	Bootstrapped $P(0,T)$	$\theta(t)$	Makes the model no-arbitrage and curve-consistent
Future dynamics	Interest-rate option premiums	a, σ	Controls mean reversion and uncertainty

```
checks = {
    "Bootstrap reprices input bonds": np.max(np.abs(repriced_bonds - clean_prices)) < 1e-10,
    "Discount factors are positive": np.all(bootstrapped_discounts > 0),
    "Discount factors decrease with maturity": np.all(np.diff(bootstrapped_discounts) < 0),
    "Hull-White matches today's discount curve": np.max(
        np.abs(hw_discounts_today - bootstrapped_discounts)
    ) < 1e-10,
    "Two vanilla-bond pricing methods agree": abs(
        hw_curve_price - direct_curve_price
    ) < 1e-10,
}

display(pd.Series(checks, name="passed").to_frame())
assert all(checks.values())
```

	passed
Bootstrap reprices input bonds	True
Discount factors are positive	True
Discount factors decrease with maturity	True
Hull–White matches today's discount curve	True
Two vanilla-bond pricing methods agree	True

12 Takeaways

- Market bond prices can be converted into discount factors by bootstrapping.

- Hull–White does not fit a whole curve using only two constants. The function $\theta(t)$ fits today's term structure; a and σ describe future dynamics.
- Option prices provide information about those dynamics that vanilla bond prices do not.
- At a future date, each one-factor Hull–White short-rate state produces a complete yield curve and a value for the remaining cash flows.
- A vanilla bond has the same time-0 value under calibrated Hull–White and direct curve discounting. The model earns its keep when future rates affect valuation or cash flows.

Simplifications

Real desks use multiple instrument types, day-count conventions, accrued interest, bid/ask spreads, smooth curve interpolation, and robust numerical optimizers. The demo deliberately removes those layers so the economic logic stays visible.